Sprawozdanie z Projektu: Lista Dwukierunkowa na Stercie z Wykorzystaniem Wzorców Projektowych oraz Narzędzi Doxygen i Git

Krzysztof Zdzieszyński
Akademia Nauk Stosowanych w Nowym Sączu

20 października 2026

Spis treści

1 Rozdział 1: Ogólne określenie wymagań **2**
1.1 Wymagania funkcjonalne i programistyczne 2
1.2 Wymagania dotyczące narzędzi i dokumentacji 2

2 Rozdział 2: Analiza problemu **3**
2.1 Struktura danych listy dwukierunkowej 3
2.2 Wzorce projektowe 3
2.3 Systemy wspierające: Git oraz Doxygen 3

3 Rozdział 3: Projektowanie **4**

4 Rozdział 4: Implementacja **5**
4.1 Kluczowe metody klasy DoublyLinkedList 5
4.2 Repozytorium Git oraz historia commitów 5
4.3 Weryfikacja scenariuszy w Git 6
4.4 Generowanie dokumentacji Doxygen 6

5 Rozdział 5: Wnioski końcowe **7**

A Załącznik: Dokumentacja techniczna Doxygen **8**

1 Rozdział 1: Ogólne określenie wymagań

Celem niniejszego projektu było opracowanie i przetestowanie modułowej implementacji dwukierunkowej listy wiązanej działającej na stercie w języku C++, z uwzględnieniem dobrych praktyk inżynierii oprogramowania oraz nowoczesnych narzędzi deweloperskich.

1.1 Wymagania funkcjonalne i programistyczne

- **Modułowość kodu:** Podział struktury na osobne pliki nagłówkowe (`Node.h`, `Iterator.h`, `DoublyLinkedList.h`, `ListFactory.h`).
- **Wzorce projektowe:** Zastosowanie wzorca *Factory* (Fabryka) do bezpiecznego tworzenia obiektów listy na stercie oraz wzorca *Iterator* do sekwencyjnej nawigacji po węzłach.
- **Operacje na liście:**
  - Dodawanie elementu na początek, na koniec oraz pod wskazany indeks.
  - Usuwanie elementu z początku, z końca oraz spod wskazanego indeksu.
  - Wyświetlanie całej listy od początku oraz w odwrotnej kolejności.
  - Swobodne przechodzenie do następnego oraz poprzedniego elementu przy użyciu iteratora.
  - Czyszczenie całej listy i poprawne zwalnianie pamięci sterty.

1.2 Wymagania dotyczące narzędzi i dokumentacji

- **Doxygen:** Automatyczne wyciąganie komentarzy z kodu źródłowego i wygenerowanie dokumentacji technicznej w formacie LaTeX kompilowanej do pliku `refman.pdf`.
- **System kontroli wersji Git:** Rejestracja historii zmian w serwisie GitHub obejmująca:
  - Wykonanie co najmniej 5 commitów.
  - Przetestowanie cofnięcia się o dwa commity.
  - Usunięcie wskazanego commita z historii.
  - Synchronizację pracy między dwiema różnymi lokalizacjami na komputerze.
  - Symulację usunięcia pliku z projektu, wywołanie błędu kompilacji i przywrócenie poprawnej wersji z repozytorium GitHub.

2 Rozdział 2: Analiza problemu

2.1 Struktura danych listy dwukierunkowej

Lista dwukierunkowa to liniowa struktura danych składająca się z węzłów alokowanych dynamicznie na stercie. Każdy węzeł zawiera wartość przechowaną oraz dwa wskaźniki: na następny (`next`) i poprzedni (`prev`) element. Taka architektura pozwala na wykonywanie operacji na obu krańcach ze złożonością czasową $O(1)$.

2.2 Wzorce projektowe

1. **Wzorzec Factory (Fabryka):** Klasa `ListFactory` umożliwia enkapsulację procesu tworzenia listy na stercie (`new DoublyLinkedList<T>()`). Chroni to kod kliencki przed bezpośrednim zarządzeniem pamięcią przy tworzeniu instancji.

2. **Wzorzec Iterator:** Klasa `ListIterator` abstrahuje proces przechodzenia po strukturze danych. Pozwala to na dostęp do elementów bez konieczności ujawniania wewnętrznych połączeń wskaźnikowych.

2.3 Systemy wspierające: Git oraz Doxygen

System **Git** umożliwia pracę wieloetapową, śledzenie zmian w plikach oraz odzyskiwanie spójnego stanu aplikacji po uszkodzeniach. **Doxygen** natomiast analizuje specjalne bloki komentarzy w kodzie źródłowym i generuje pełny wykaz klas, metod oraz pól w formacie LaTeX, z którego tworzony jest plik PDF.

3 Rozdział 3: Projektowanie

Projekt został zaprojektowany z zachowaniem zasady pojedynczej odpowiedzialności (SRP), gdzie każda klasa znajduje się w osobnym pliku nagłówkowym:

- `Node.h` – generyczna struktura pojedynczego węzła przechowywana na stercie.
- `Iterator.h` – klasa `ListIterator` realizująca przeglądanie elementów w przód i w tył.
- `DoublyLinkedList.h` – główna klasa zarządzająca węzłami na stercie, realizująca operacje wstawiania, usuwania i wyświetlania.
- `ListFactory.h` – klasa fabryki odpowiedzialna za alokację listy.
- `main.cpp` / `test_lista_dwukierunkowa.cpp` – moduły testowe sprawdzające poprawne działanie wszystkich metod.

4 Rozdział 4: Implementacja

4.1 Kluczowe metody klasy DoublyLinkedList

Poniżej przedstawiono implementację metody dodawania elementu pod wskazany indeks z zachowaniem poprawności wskaźników `next` oraz `prev`:

Listing 1: Dodawanie elementu pod podany indeks

```
void addAt(int index, const T& val) {
    if (index < 0 || index > count) {
        throw std::out_of_range("Indeks poza zakresem!");
    }
    if (index == 0) {
        addFront(val);
    } else if (index == count) {
        addBack(val);
    } else {
        Node<T>* current = getNodeAt(index);
        Node<T>* newNode = new Node<T>(val);
        Node<T>* prevNode = current->prev;

        prevNode->next = newNode;
        newNode->prev = prevNode;
        newNode->next = current;
        current->prev = newNode;
        count++;
    }
}
```

4.2 Repozytorium Git oraz historia commitów

- **Adres repozytorium GitHub:** `https://github.com/KrzysztofZdzieszynski/lista-dwukierunkowa`

W ramach pracy z systemem kontroli wersji rejestrowano postępy w repozytorium. Poniżej przedstawiono pełny log z historii commitów (`git log -oneline -graph -all`):

Listing 2: Rzeczywista historia commitów z systemu Git

```
* da16ced (HEAD -> main, origin/main) Commit 5: Dodanie wzorca
   Factory, wyswietlania wstecz oraz pliku main.cpp
* 2b3b477 Commit 4: Implementacja wzorca Iteratora oraz integracja
   z klasa listy
* 43acabd Commit 3: Dodanie metod removeFront, removeBack oraz
   czyszczenia listy
* 181c786 Commit 2: Dodanie metod addFront, addBack oraz
   wyswietlania listy
* e3a06e8 Commit 1: Dodanie struktury Node i bazowego szkieletu
   klasy listy
```

4.3 Weryfikacja scenariuszy w Git

1. Utworzono 5 chronologicznych commitów dokumentujących postęp prac.
2. Przeprowadzono testowy powrót w historii o dwa commity wstecz za pomocą polecenia `git reset -hard HEAD`.
3. Sklonowano repozytorium do drugiego katalogu na dysku, wprowadzono zmiany, wysłano je na serwer zdalny (`git push`) i zsynchronizowano w pierwotnej lokalizacji przy użyciu `git pull`.
4. Celowo usunięto plik `ListFactory.h`, wywołując błąd kompilacji, a następnie przywrócono go poleceniem `git checkout ListFactory.h`.

4.4 Generowanie dokumentacji Doxygen

Plik konfiguracyjny `Doxyfile` został wygenerowany poleceniem `doxygen -g Doxyfile` z włączoną opcją `GENERATE_LATEX = YES`. Wynikowe pliki trafiły do katalogu `latex/`, po czym zostały skompilowane poleceniem `pdflatex refman.tex` do pliku `refman.pdf`.

5 Rozdział 5: Wnioski końcowe

W wyniku realizacji projektu powstała w pełni funkcjonalna implementacja dwukierunkowej listy wiązanej na stercie w języku C++. Zastosowanie wzorców projektowych *Factory* i *Iterator* zwiększyło czytelność oraz elastyczność kodu.

Integracja generatora **Doxygen** z systemem **LaTeX** pozwoliła na sprawną automatyzację powstawania dokumentacji technicznej API. Ćwiczenia praktyczne z narzędziem **Git** pozwoliły na opanowanie kluczowych mechanizmów kontroli wersji, wycofywania błędnych zmian oraz pracy z repozytorium zdalnym GitHub.

A Załącznik: Dokumentacja techniczna Doxygen

Poniżej dołączono dokumentację techniczną wygenerowaną automatycznie przez program Doxygen do pliku PDF.

My Project

Generated by Doxygen 1.10.0

contentsline/text/before4 01 Class Index11 **Class Index**contentsline/text/after4 01 Class Index1toc/leaders/beforetoc/leaders **toc/leaderstoc/leaders/after**contentsline/page/before4 01 **1**contentsline/page/after4 01

contentsline/text/before4 11.1 Class List11.1 Class Listcontentsline/text/after4 11.1 Class List1toc/leaders/beforetoc/leaders toc/leaderstoc/leaders/aftercontentsline/page/before4 11 1contentsline/page/after4 11

contentsline/text/before4 02 File Index32 **File Index**contentsline/text/after4 02 File Index3toc/leaders/beforetoc/leaders **toc/leaderstoc/leaders/after**contentsline/page/before4 03 **3**contentsline/page/after4 03

contentsline/text/before4 12.1 File List32.1 File Listcontentsline/text/after4 12.1 File List3toc/leaders/beforetoc/leaders toc/leaderstoc/leaders/aftercontentsline/page/before4 13 3contentsline/page/after4 13

contentsline/text/before4 03 Class Documentation53 **Class Documentation**contentsline/text/after4 03 Class Documentation5toc/leaders/beforetoc/leaders **toc/leaderstoc/leaders/after**contentsline/page/before4 05 **5**contentsline/page/after4 05

contentsline/text/before4 13.1 DoublyLinkedList< T > Class Template Reference53.1 DoublyLinkedList< T > Class Template Referencecontentsline/text/after4 13.1 DoublyLinkedList< T > Class Template Reference5toc/leaders/beforetoc/leaders . . toc/leaderstoc/leaders/aftercontentsline/page/before4 15 5contentsline/page/after4 15

contentsline/text/before4 13.2 ListFactory< T > Class Template Reference53.2 ListFactory< T > Class Template Referencecontentsline/text/after4 13.2 ListFactory< T > Class Template Reference5toc/leaders/beforetoc/leaders toc/leaderstoc/leaders/aftercontentsline/page/before4 15 5contentsline/page/after4 15

contentsline/text/before4 13.3 ListIterator< T > Class Template Reference63.3 ListIterator< T > Class Template Referencecontentsline/text/after4 13.3 ListIterator< T > Class Template Reference6toc/leaders/beforetoc/leaders toc/leaderstoc/leaders/aftercontentsline/page/before4 16 6contentsline/page/after4 16

contentsline/text/before4 13.4 Node< T > Struct Template Reference63.4 Node< T > Struct Template Referencecontentsline/text/after4 13.4 Node< T > Struct Template Reference6toc/leaders/beforetoc/leaders toc/leaderstoc/leaders/aftercontentsline/page/before4 16 6contentsline/page/after4 16

contentsline/text/before4 04 File Documentation74 **File Documentation**contentsline/text/after4 04 File Documentation7toc/leaders/beforetoc/leaders **toc/leaderstoc/leaders/after**contentsline/page/before4 07 **7**contentsline/page/after4 07

contentsline/text/before4 14.1 DoublyLinkedList.h74.1 DoublyLinkedList.hcontentsline/text/after4 14.1 DoublyLinkedList.h7toc/leaders/beforetoc/leaderstoc/leaderstoc/leaders/aftercontentsline/page/before4 17 7contentsline/page/after4 17

contentsline/text/before4 14.2 Iterator.h84.2 Iterator.hcontentsline/text/after4 14.2 Iterator.h8toc/leaders/beforetoc/leaders toc/leaderstoc/leaders/aftercontentsline/page/before4 18 8contentsline/page/after4 18

contentsline/text/before4 14.3 ListFactory.h94.3 ListFactory.hcontentsline/text/after4 14.3 ListFactory.↩h9toc/leaders/beforetoc/leaders toc/leaderstoc/leaders/aftercontentsline/page/before4 19 9contentsline/page/after4 19

contentsline/text/before4 14.4 Node.h94.4 Node.hcontentsline/text/after4 14.4 Node.h9toc/leaders/beforetoc/leaders toc/leaderstoc/leaders/aftercontentsline/page/before4 19 9contentsline/page/after4 19

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

contentsline/text/before4 1DoublyLinkedList< T > 5DoublyLinkedList< T > contentsline/text/after4 1DoublyLinkedList< T > 5toc/leaders/beforetoc/leaderstoc/leaderstoc/leaders/aftercontentsline/page/before4 15 5contentsline/page/after4 15

contentsline/text/before4 1ListFactory< T > 5ListFactory< T > contentsline/text/after4 1ListFactory< T > 5toc/leaders/beforetoc/leaders toc/leaderstoc/leaders/aftercontentsline/page/before4 15 5contentsline/page/after4 15

contentsline/text/before4 1ListIterator< T > 6ListIterator< T > contentsline/text/after4 1ListIterator< T > 6toc/leaders/beforetoc/leaders toc/leaderstoc/leaders/aftercontentsline/page/before4 16 6contentsline/page/after4 16

contentsline/text/before4 1Node< T > 6Node< T > contentsline/text/after4 1Node< T > 6toc/leaders/beforetoc/leaders toc/leaderstoc/leaders/aftercontentsline/page/before4 16 6contentsline/page/after4 16

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

contentsline/text/before4 1DoublyLinkedList.h 7DoublyLinkedList.h contentsline/text/after4 1DoublyLinkedList.h 7toc/leaders/beforetoc/leaders toc/leaderstoc/leaders/aftercontentsline/page/before4 17 7contentsline/page/after4 17
contentsline/text/before4 1Iterator.h 8Iterator.h contentsline/text/after4 1Iterator.h 8toc/leaders/beforetoc/leaders toc/leaderstoc/leaders/aftercontentsline/page/before4 18 8contentsline/page/after4 18
contentsline/text/before4 1ListFactory.h 9ListFactory.h contentsline/text/after4 1ListFactory.h 9toc/leaders/beforetoc/leaders toc/leaderstoc/leaders/aftercontentsline/page/before4 19 9contentsline/page/after4 19
contentsline/text/before4 1Node.h 9Node.h contentsline/text/after4 1Node.h 9toc/leaders/beforetoc/leaders toc/leaderstoc/leaders/aftercontentsline/page/before4 19 9contentsline/page/after4 19

Chapter 3

Class Documentation

3.1 DoublyLinkedList< T > Class Template Reference

Public Member Functions

- bool **isEmpty** () const
- void **addFront** (const T &val)
- void **addBack** (const T &val)
- void **removeFront** ()
- void **removeBack** ()
- void **clear** ()
- void **display** () const
- ListIterator< T > **getIterator** () const
- void **displayReverse** () const

The documentation for this class was generated from the following file:

- DoublyLinkedList.h

3.2 ListFactory< T > Class Template Reference

Static Public Member Functions

- static DoublyLinkedList< T > * **createList** ()

The documentation for this class was generated from the following file:

- ListFactory.h

3.3 ListIterator< T > Class Template Reference

Public Member Functions

- **ListIterator** (Node< T > *startNode)
- bool **hasNext** () const
- bool **hasPrev** () const
- T **next** ()
- T **prev** ()
- T **getCurrent** () const
- bool **isValid** () const

The documentation for this class was generated from the following file:

- Iterator.h

3.4 Node< T > Struct Template Reference

Public Member Functions

- **Node** (const T &wartosc)

Public Attributes

- T **data**
- Node * **next**
- Node * **prev**

The documentation for this struct was generated from the following file:

- Node.h

Chapter 4

File Documentation

4.1 DoublyLinkedList.h

```
00001 #ifndef DOUBLY_LINKED_LIST_H
00002 #define DOUBLY_LINKED_LIST_H
00003
00004 #include <iostream>
00005 #include "Node.h"
00006 #include "Iterator.h"
00007
00008 // Główna klasa listy dwukierunkowej działająca na stercie
00009 template <typename T>
00010 class DoublyLinkedList {
00011 private:
00012     Node<T>* head; // początek listy
00013     Node<T>* tail; // koniec listy
00014     int count;     // licznik elementów
00015
00016 public:
00017     // Konstruktor - tworzy pustą listę
00018     DoublyLinkedList() {
00019         head = nullptr;
00020         tail = nullptr;
00021         count = 0;
00022     }
00023
00024     // Destruktor - czyszczenie pamięci
00025     ~DoublyLinkedList() {
00026         clear();
00027     }
00028
00029     // Sprawdzenie czy lista jest pusta
00030     bool isEmpty() const {
00031         return head == nullptr;
00032     }
00033
00034     // Dodanie elementu na początek listy
00035     void addFront(const T& val) {
00036         Node<T>* newNode = new Node<T>(val);
00037         if (isEmpty()) {
00038             head = tail = newNode;
00039         } else {
00040             newNode->next = head;
00041             head->prev = newNode;
00042             head = newNode;
00043         }
00044         count++;
00045     }
00046
00047     // Dodanie elementu na koniec listy
00048     void addBack(const T& val) {
00049         Node<T>* newNode = new Node<T>(val);
00050         if (isEmpty()) {
00051             head = tail = newNode;
00052         } else {
00053             tail->next = newNode;
00054             newNode->prev = tail;
00055             tail = newNode;
00056         }
00057         count++;
00058     }
```

```
00059
00060     // Usunięcie elementu z początku listy
00061     void removeFront() {
00062         if (isEmpty()) return;
00063         Node<T>* temp = head;
00064         head = head->next;
00065         if (head != nullptr) {
00066             head->prev = nullptr;
00067         } else {
00068             tail = nullptr; // lista stała się pusta
00069         }
00070         delete temp;
00071         count--;
00072     }
00073
00074     // Usunięcie elementu z końca listy
00075     void removeBack() {
00076         if (isEmpty()) return;
00077         Node<T>* temp = tail;
00078         tail = tail->prev;
00079         if (tail != nullptr) {
00080             tail->next = nullptr;
00081         } else {
00082             head = nullptr; // lista stała się pusta
00083         }
00084         delete temp;
00085         count--;
00086     }
00087
00088     // Czyszczenie całej listy
00089     void clear() {
00090         while (!isEmpty()) {
00091             removeFront();
00092         }
00093     }
00094
00095     // Wyświetlanie listy od początku
00096     void display() const {
00097         Node<T>* current = head;
00098         std::cout « "Lista: ";
00099         while (current != nullptr) {
00100             std::cout « current->data « " ";
00101             current = current->next;
00102         }
00103         std::cout « std::endl;
00104     }
00105     // Zwraca iterator początkowy listy
00106     ListIterator<T> getIterator() const {
00107         return ListIterator<T>(head);
00108     }
00109     // Wyświetlanie listy od końca (w odwrotnej kolejności)
00110     void displayReverse() const {
00111         Node<T>* current = tail;
00112         std::cout « "Lista od tylu: ";
00113         while (current != nullptr) {
00114             std::cout « current->data « " ";
00115             current = current->prev;
00116         }
00117         std::cout « std::endl;
00118     }
00119 };
00120
00121 #endif
```

4.2 Iterator.h

```
00001 #ifndef ITERATOR_H
00002 #define ITERATOR_H
00003
00004 #include "Node.h"
00005
00006 // Wzorzec Iteratora do przechodzenia po liście dwukierunkowej
00007 template <typename T>
00008 class ListIterator {
00009 private:
00010     Node<T>* current; // aktualny węzeł, na którym stoi iterator
00011
00012 public:
00013     // Konstruktor przyjmujący początkowy węzeł
00014     ListIterator(Node<T>* startNode) {
00015         current = startNode;
00016     }
00017
```

```
00018     // Sprawdzenie czy jest następny element
00019     bool hasNext() const {
00020         return current != nullptr && current->next != nullptr;
00021     }
00022
00023     // Sprawdzenie czy jest poprzedni element
00024     bool hasPrev() const {
00025         return current != nullptr && current->prev != nullptr;
00026     }
00027
00028     // Przejście do następnego elementu i zwrócenie jego wartości
00029     T next() {
00030         if (current != nullptr && current->next != nullptr) {
00031             current = current->next;
00032         }
00033         return current ? current->data : T();
00034     }
00035
00036     // Przejście do poprzedniego elementu i zwrócenie jego wartości
00037     T prev() {
00038         if (current != nullptr && current->prev != nullptr) {
00039             current = current->prev;
00040         }
00041         return current ? current->data : T();
00042     }
00043
00044     // Pobranie wartości aktualnego elementu
00045     T getCurrent() const {
00046         return current ? current->data : T();
00047     }
00048
00049     // Sprawdzenie czy iterator wskazuje na poprawny element
00050     bool isValid() const {
00051         return current != nullptr;
00052     }
00053 };
00054
00055 #endif
```

4.3 ListFactory.h

```
00001 #ifndef LIST_FACTORY_H
00002 #define LIST_FACTORY_H
00003
00004 #include "DoublyLinkedList.h"
00005
00006 // Wzorzec Projektowy Factory (Fabryka) do tworzenia listy na stercie
00007 template <typename T>
00008 class ListFactory {
00009 public:
00010     static DoublyLinkedList<T>* createList() {
00011         return new DoublyLinkedList<T>();
00012     }
00013 };
00014
00015 #endif
```

4.4 Node.h

```
00001 #ifndef NODE_H
00002 #define NODE_H
00003
00004 // Pojedynczy element listy dwukierunkowej
00005 template <typename T>
00006 struct Node {
00007     T data;         // wartość
00008     Node* next;     // wskaźnik na następny element
00009     Node* prev;     // wskaźnik na poprzedni element
00010
00011     // Konstruktor do tworzenia węzła
00012     Node(const T& wartosc) {
00013         data = wartosc;
00014         next = nullptr;
00015         prev = nullptr;
00016     }
00017 };
00018
00019 #endif
```